

Interview Study Narratives – Elastic Detection Engineering

Case Study 1: Suspicious PowerShell Flags (EQL)

****Summary:**** Developed and validated a detection rule to identify potentially malicious PowerShell activity involving obfuscation and encoded commands.

****Situation:**** Needed to detect obfuscated PowerShell executions (e.g., `-EncodedCommand`, `-NoProfile`). ****Task:**** Create an Elastic EQL rule to capture process-level telemetry aligned to MITRE ATT&CK; T1059.001 (Execution). ****Action:**** • Identified that ``process.command_line`` data was missing due to incorrect Fleet enrollment. • Uninstalled and re-enrolled the Elastic Agent, then enabled Elastic Defend to restore endpoint process telemetry. • Validated telemetry in Discover (``endpoint.events.process``) before rule creation. • Built and tuned the EQL rule: ``process where process.name : ("powershell.exe", "pwsh.exe") and (process.command_line : "*-enc*" or process.command_line : "*-EncodedCommand*" or process.command_line : "*-NoProfile*" or process.command_line : "*-nop*")`` • Resolved indexing mismatch by searching ``kibana.alert.rule.name`` instead of ``rule.name``. ****Result:**** Achieved reliable alerts on encoded and no-profile PowerShell executions with minimal false positives.

****MITRE ATT&CK; Mapping:**** Execution → Command and Scripting Interpreter: PowerShell (T1059.001)

****Common Interview Questions & Responses:**** 1. **How did you verify telemetry before writing the rule?** Verified ``process.command_line`` in Discover under ``endpoint.events.process`` using test PowerShell executions. 2. **What was the root cause of missing data?** Agent lacked Elastic Defend; process telemetry wasn't being forwarded. Re-enrollment fixed it. 3. **How did you test the rule?** Ran PowerShell with ``-EncodedCommand`` and ``-NoProfile`` flags; confirmed alerts via ``kibana.alert.rule.name``. 4. **How would you reduce false positives?** Scope rule to user-owned processes or exclude known administrative scripts. 5. **What did you learn?** Detection logic is meaningless without telemetry validation; rule development must start with ingestion verification.

****Challenges & Lessons Learned:**** Faced multiple false starts due to telemetry gaps, index misconfigurations, and Elastic UI inconsistencies. Learned to diagnose ingestion issues through Discover rather than assuming query errors. Re-enrollment and Elastic Defend integration restored full visibility and confirmed the value of verifying data pipelines before writing detections.

Case Study 2: Failed Logon Burst (4625)

****Summary:**** Implemented a detection rule to identify bursts of failed Windows logon attempts, signaling potential brute-force or credential stuffing attacks.

****Situation:**** Needed to detect multiple failed logon events across Windows systems. ****Task:**** Configure telemetry collection for Security logs and create an EQL/KQL rule to alert on excessive failed

logons (Event ID 4625). ****Action:**** • Diagnosed missing Security event telemetry in default Windows integration. • Resolved by adding the ****System Integration**** and enabling the ``windows.security`` log stream. • Verified event ingestion by searching ``data_stream.dataset: "system.security" AND winlog.event_id: 4625``. • Built and tested the rule: threshold-based logic counting >20 failed logons in 15 minutes. • Validated alert generation and captured alert table and detail screenshots. ****Result:**** Achieved reliable detection of failed logon bursts mapped to MITRE ATT&CK; T1110 (Credential Access).

****MITRE ATT&CK; Mapping:**** Credential Access → Brute Force (T1110)

****Common Interview Questions & Responses:**** 1. **How did you enable failed logon telemetry?** Added the System integration and enabled ``windows.security`` logs for event ID 4625. 2. **Why didn't you use the Windows integration?** It lacked Security channel coverage in this version; System integration provided full visibility. 3. **How did you test it?** Simulated failed logons via login screen and ``runas.exe`` command; validated logs in Discover. 4. **What was the biggest challenge?** Discovering Elastic's inconsistent UI between integrations; required manual event stream inspection. 5. **What did you learn?** Validating data ingestion before detection building is crucial; even correct rules won't fire without proper sources.

****Challenges & Lessons Learned:**** This phase required differentiating between process events and security log events, revealing how Elastic handles distinct telemetry pipelines. Adding the System integration for Security logs proved essential for detecting logon failures. Reinforced the importance of confirming dataset scope and event types before correlation or threshold logic creation.

End of Interview Study Narratives